HOTEL RESERVATION SYSTEM

Claude Code Engineering Guide

Project	Hotel Reservation System
Document	Claude Code Engineering Guide
Document ID	ENG-001
Version	1.0
Status	Active
Owner	Engineering
Last Updated	2026-09-13

Official readable project artifact. Editable source is maintained under `docs/_source/`.

Contents

- Document control
- Purpose
- Locked engineering constraints
- Modular Monolith rules
- Scope discipline
- Security rules
- Reservation integrity rules
- Testing behavior
- Working with project documentation
- Standard Claude Code task prompt template
- Completion-report standard
- Review responsibility
- Current bounded task reference

Document control

Version	Date	Change	Author
1.0	2026-09-13	Initial AI-assisted engineering guardrails and task prompt standard	Engineering

Purpose

Claude Code may assist the developer with implementation, review, testing, and repository changes, but the **developer remains responsible for understanding, validating, and accepting all code**. Claude Code is not the project owner and must not expand scope, change architecture, or introduce technologies merely because they are convenient.

The official project context is the versioned documentation under `docs/`, especially the Project Handbook, Product Backlog, System Architecture, ADR-001, and the active Sprint Plan. Claude Code should inspect the repository and these artifacts rather than relying on chat history.

Locked engineering constraints

Area	Constraint
Repository	npm workspaces monorepo
Frontend app	`apps/web`
Backend app	`apps/api`
Language	TypeScript for frontend and backend
Frontend	Next.js
Backend	Node.js + Express
API	REST; business namespace `/api/v1`
Database	PostgreSQL
Hosting	Railway
Backend architecture	Feature-first Modular Monolith
Database topology	One shared PostgreSQL database
Deployment topology	One frontend app + one backend app; no microservices
Currency	EGP

Area	Constraint
Booking	Guest books Room Type; backend allocates physical Room
Reservation interval	`[check-in, check-out)`
Payment	Pay at Hotel / Cash on Arrival

Modular Monolith rules

Claude Code must preserve the following:

- Business code is grouped by capability under `apps/api/src/modules/`.
- There is no global mixed-domain business structure such as root-level `controllers/`, `services/`, and `repositories/` folders.
- A module may contain routes, controllers, services, repositories, schemas, or types only when the current requirement needs them.
- Modules do not freely import another module's internal files.
- Cross-module behavior is exposed intentionally through a public capability owned by the source module.
- `shared/` is for genuine cross-cutting technical concerns only; business logic remains in its owning module.
- Do not create one internal npm package per module.
- Do not introduce microservices, event buses, CQRS, generic base repositories, DI frameworks, or domain-event infrastructure without explicit approval and a concrete requirement.

Scope discipline

For every task, Claude Code must:

1. Read the current Sprint task and relevant architecture documentation.
2. Inspect the existing repository before proposing or changing files.
3. Work on **one bounded task at a time**.
4. Respect the current Sprint boundary and selected PBIs.
5. Avoid implementing future PBIs "while here."
6. Avoid unrelated refactoring, renaming, formatting churn, or dependency upgrades.
7. Avoid speculative abstraction and placeholder directories/files.
8. Avoid adding a new framework, library, service, or build tool without explicit approval when an existing/simple approach is sufficient.
9. Preserve module boundaries and repository ownership of dependencies.
10. Stop and report if the requested task would require a material architecture decision not already approved.

Security rules

Claude Code must never:

- Commit secrets, `.env` credentials, Railway tokens, database passwords, auth secrets, or email-provider credentials.
- Put server credentials into client-exposed Next.js environment variables.
- Trust client-side booking totals, nightly prices, physical room IDs, or cached availability as authoritative.
- Bypass backend validation for business-critical behavior.
- Weaken database-level reservation integrity to make tests pass.
- Log sensitive guest data unnecessarily.
- Expose password hashes, internal room allocation, stack traces, or database details through public responses.
- Disable security controls globally merely to simplify local development.

If Claude Code encounters a security/integrity tradeoff, it should surface the issue rather than silently weaken the design.

Reservation integrity rules

When reservation work begins, Claude Code must treat these rules as non-negotiable unless the architecture document is formally revised:

- Stay interval is `[check-in, check-out)`.
- Pending reservations consume inventory immediately.
- One reservation consumes one physical room.
- Public guests select room type, not physical room.
- Availability is rechecked during reservation creation.
- Physical room allocation is protected in PostgreSQL transaction/concurrency logic.
- Overlapping inventory-consuming reservations for one physical room are prevented at the database level.
- Nights and price are calculated server-side.
- Accepted pricing is snapshotted on the reservation.
- Public booking responses never expose the internal room number.
- A concurrency test for the one remaining room must result in exactly one successful overlapping booking.

Testing behavior

Claude Code should add or update tests **when the current task requires them**, not create a large speculative test framework in advance.

Priority for Sprint 1 is:

- database/domain integration tests for date and overlap behavior,
- pricing and historical snapshot tests,
- concurrency tests,
- API integration tests,
- focused frontend tests where they provide value,
- end-to-end proof of the booking journey.

Tests must validate behavior, not merely implementation details.

Working with project documentation

- Treat versioned PDFs as the official readable artifacts.
- Treat Markdown under `docs/_source/` as the editable source used to regenerate the PDFs.
- Do not silently rewrite architecture/product decisions while implementing a task.
- If implementation reveals a material mismatch, report it and propose a documentation change for developer approval.
- Sprint task status changes should be reflected in the Sprint Plan when the developer accepts the change.

Standard Claude Code task prompt template

Use the following structure for each bounded task.

```
PROJECT
[Project name and one-paragraph context]

SOURCE OF TRUTH
[List the project documents Claude should read before changing code]

LOCKED ARCHITECTURE
[List only constraints relevant to this task]

CURRENT SPRINT / TASK
[Sprint number, task ID, objective, status]

WHY THIS TASK EXISTS
[Value / dependency / risk being reduced]

ALLOWED SCOPE
[Explicit files, folders, behavior, and tooling allowed]

EXPLICIT EXCLUSIONS
[Features, PBIs, technologies, refactors, and folders that must not be introduced]

ACCEPTANCE CRITERIA
[Observable definition of success]

TESTS / CHECKS REQUIRED
[Commands and scenarios that must be run]

SECURITY / QUALITY RULES
[Task-specific guardrails]

EXPECTED WORKFLOW
1. Inspect repository and relevant docs first.
2. Summarize current state and proposed minimal changes.
3. Make only the approved/bounded changes.
4. Run required checks/tests.
5. Do not expand scope to fix unrelated issues; report them separately.

COMPLETION REPORT
- Files changed
- What was implemented
- Tests/checks run
- Results
- Architectural decisions made (ideally none unless already approved)
- Remaining concerns / blockers
- Anything intentionally not implemented because it is outside scope
```

Completion-report standard

Every Claude Code task completion should clearly state:

1. **Files changed** - added/modified/deleted paths.
2. **What was implemented** - behavior, not marketing language.
3. **Tests/checks run** - exact commands or scenarios where practical.
4. **Results** - pass/fail and important warnings.

5. **Architectural decisions made** - call out any choice that may require documentation; avoid inventing new architecture silently.
6. **Remaining concerns** - blockers, uncertain behavior, technical debt introduced by the bounded task.
7. **Intentionally not implemented** - confirm important exclusions stayed out of scope.

Review responsibility

The developer should not accept generated changes solely because tests are green. Before marking a task Done, the developer should review:

- architectural fit,
- module boundaries,
- error handling,
- security implications,
- test relevance,
- dependency additions,
- configuration/secrets,
- unnecessary abstraction,
- and whether the implementation actually satisfies the Sprint task rather than merely compiling.

Current bounded task reference

The first Claude Code execution prompt is maintained as:

`docs/04-engineering/prompts/Sprint_01_Task_01_Claude_Code_Prompt_v1.0.md`

It is intentionally limited to the repository/application skeleton and health integration. It forbids PostgreSQL, ORM, rooms, reservations, authentication, packages, email, media, Railway deployment work, CI workflow implementation, and future shared abstractions.